

LoopThief — Handoff Document

1. Project Overview

LoopThief is a browser-based MPC-style sampler/sequencer workstation inspired primarily by MPC2000XL workflow.

Core goals:

- authentic MPC-style workflow
- hardware-like UI and interaction
- fast sampling/chopping/sequencing
- bank/program based sequencing
- real-time performance feel
- no fake UI controls

Current stage:

- prototype evolving into real architecture
- major audio/sequencer foundations already exist
- many screens operational
- several systems still partial or placeholder

Current priorities: 1. stabilize architecture 2. preserve MPC workflow authenticity 3. remove fake systems/placeholders 4. improve sequencing/program behavior 5. project save/load 6. real FX later

Already reliable:

- sample import/export WAV
- recording workflow
- chop workflow
- sequencing basics
- track mute
- song mode basics
- bank-aware playback
- real program ownership
- tune/fine tune
- filter routing
- mix screen basics
- metronome groundwork
- 16 levels groundwork

Still WIP:

- save/load full project pipeline
- FX engine
- MIDI engine
- advanced automation
- timestretch

- proper performance mode
 - advanced note repeat
 - full undo/redo
 - true realtime resampling pipeline
-

2. Audio Architecture

Sample Pipeline

General flow:

WAV import -> sample library/runtime registry -> program pad assignment -> playback voice -> mixer -> output

Samples are runtime-owned PCM buffers.

PCM buffers are intentionally NOT fully serialized yet.

Main concepts:

- sample metadata persists
- PCM/audio buffers remain runtime-only
- programs own assignments
- playback resolves through program + bank + pad identity

Playback Flow

Typical playback path:

Pad trigger or StepEvent -> resolve program -> resolve bank + pad -> fetch assigned sample -> apply tune/fine tune -> optional filter routing -> mixer gain/pan -> master output

Tune / Fine Tune

Pitch logic:

$\text{playbackRate} = 2^{((\text{semitones} + \text{cents} / 100) / 12)}$

Applied consistently to:

- live pad hits
- sequence playback
- note repeat
- 16 levels playback

Filter Routing

Implemented using WebAudio BiquadFilterNode.

Supported types currently:

- OFF
- LOWPASS
- HIGHPASS
- BANDPASS

When OFF:

- filter node bypassed entirely

Filter updates should affect active voices in realtime.

Mixer

Current mixer:

- per-pad volume
- pan
- mute/solo
- smoothed activity meters

Meters were rewritten to avoid chaotic flickering.

Recording

Recording supports:

- system audio capture
- monitor
- sample save
- waveform preview

Current limitation: Browser capture permission popup appears repeatedly due to browser security.

Potential future solutions:

- Electron/native shell
 - WASAPI loopback
 - VB-Cable support
 - persistent capture device routing
-

3. Sequencer Architecture

Timing System

Uses PPQ-based timing.

Current basis: 96 PPQ.

Positive timing offsets:

- delayed from current step

Negative timing offsets:

- scheduled from previous step

StepEvent Structure

Important fields:

- trackId
- padBank
- padNumber
- programId
- velocity
- duration
- probability
- timingOffset
- bar/beat/tick position

Legacy events without bank: default to bank A.

Sequencer Playback

Playback MUST resolve through event-owned context.

Critical rule: UI-visible bank must NEVER affect existing sequence playback.

Playback resolution order: 1. event.programId 2. event.padBank 3. event.padNumber

Old architecture incorrectly used current UI bank. That bug was fixed.

Velocity

Mouse/UI recording now records stable velocity 127.

Random velocity generation is discouraged unless explicitly enabled as humanization.

STEP Screen

Current STEP behavior:

- selecting event auditions it immediately
- navigating events auditions notes
- track filtering updates visible events
- track switching uses visible < > controls

STEP displays full bank identity:

- A01
- B12
- etc.

TC / Swing

Timing Correct screen exists.

Current implementation:

- quantize/grid
- swing amount
- strength

Needs more musical validation later.

Song Convert

Song convert now accumulates PPQ offsets correctly.

Old bug: all sequences collapsed onto same bars.

Current convert:

- expands repeats
- accumulates offsets
- preserves track/program/bank references

4. Pad / Bank / Program Architecture

Banks

Banks:

- A
- B
- C

- D

Important design decision: Banks are NOT a rotating next-state system.

Correct behavior:

- click A => A
- click B => B
- etc.

Old cycling behavior: A -> B -> C -> D was removed.

Full Pad Identity

Old system:

- global 0-15 pad mapping

New system:

- full bank-aware pad identity

Examples:

- A01
- B04
- D16

Visual pad feedback also had to be rewritten to respect bank identity.

Programs

Programs are now real ownership containers.

Programs own:

- pad assignments
- mixer state
- tune/fine tune
- filter params
- choke groups
- future FX params

Old architecture used global/shared assignment state. That caused sequencing corruption.

PROGRAM < > now swaps actual state.

Track Ownership

Tracks are structured objects.

Fields:

- id
- name
- programId
- mute
- solo
- type
- output

Events reference stable track identity.

5. UI / Screen Architecture

AppShell Rules

AppShell is considered protected/sacred.

Do NOT casually rewrite:

- layout editor
- window routing
- AppShell structure
- screen mounting system

Critical workflows:

- F2 layout edit
- Ctrl+S layout save must always survive.

Screen Philosophy

Screens should:

- resemble MPC workflows
- avoid fake controls
- avoid giant empty decorative UI
- prioritize usability over flashy visuals

Softkeys

Softkeys are central workflow anchors.

F1-F6 should always:

- map to real actions
- avoid placeholder behavior
- avoid misleading fake functionality

Safe Area

Multiple layout bugs happened because controls rendered behind softkey rows.

Rule: All screen content must fit above softkeys.

Use LCD safe-area constraints consistently.

6. Important Design Decisions

Why Real Program Ownership

Without program ownership:

- changing programs broke sequence playback
- banks contaminated other banks
- assignments became global chaos

Real MPC workflow requires: program-contained state.

Why Bank-Aware Sequencing

MPC sequencing fundamentally depends on pad identity.

A01 and B01 are different instruments.

Using current UI bank for playback caused catastrophic sequencing bugs.

Why Duplicate UI Was Removed

Several screens showed:

- static info AND
- editable version of same parameter

This wasted space and caused layout overflow.

Rule: If parameter is editable, show it once.

Separate PARAMS and FILTER Modes

PARAMS and FILTER are separate MPC workflows.

They should not be merged into one cluttered panel.

Fake UI Policy

Fake clickable controls are strongly discouraged.

Every clickable control should:

- do something real OR
 - be removed/disabled
-

7. Known Bugs / Limitations

Audio / FX

Still missing:

- real FX bus engine
- delay
- reverb
- flanger
- compressor
- EQ

FX SEND currently mostly visual.

Performance Screen

Performance screen partially placeholder. Needs real purpose.

Save / Load

Project serialization groundwork exists. Full restore pipeline incomplete.

PCM buffers still runtime-only.

Undo / Redo

Undo exists visually. Not yet reliable full-state history.

Note Repeat

Works from hardware/button path. Performance screen integration partially inconsistent.

Song Mode

Needs more testing with:

- repeats
- sequence transitions
- BPM changes
- mute automation

MIX

Output routing mostly cosmetic. Real multi-output routing not implemented.

CHOP

Loop BPM detection still simplistic. Needs better analysis.

Browser Recording

Browser security popup still annoying.

8. Key Files

NOTE: Paths may evolve. Always inspect before editing.

Likely key areas:

Store

`src/store/useAppStore.ts`

Main project brain. Contains:

- sequencing
- transport
- tracks
- programs
- UI state
- song mode
- project snapshot

Audio

`src/audio/`

Important areas:

- samplerEngine
- recordingCapture
- sampleLibrary
- playback routing
- filters
- mixer

Screens

src/screens/

Each major MPC screen. Examples:

- MainScreen
- ProgramScreen
- StepScreen
- ChopScreen
- MixScreen
- RecordScreen
- SongScreen

Layout

src/components/layout/

Protected area. Contains:

- AppShell
- layout editor
- LCD safe area
- layout persistence

9. Development Rules

Mandatory rules:

- inspect before editing
- avoid giant rewrites
- preserve AppShell/layout editor
- preserve Ctrl+S layout save
- preserve F2 layout edit
- preserve bank-aware sequencing
- preserve program ownership
- preserve MPC workflow authenticity
- no fake clickable UI
- keep screen workflows coherent

- npm run build after changes

Communication preference:

- concise
- direct
- technical
- no marketing tone

Architecture philosophy:

- stabilize before expanding
 - quality over flashy features
 - real systems over placeholder visuals
-

10. Recommended Next Priorities

Short Term

- polish remaining layouts
- finish real program editing
- improve song mode reliability
- proper note repeat integration
- improve metronome
- stabilize performance mode
- fix remaining placeholder actions

Medium Term

- full project save/load
- real FX engine
- MIDI support
- automation lanes
- sample browser improvements
- realtime waveform updates

Long Term

- timestretch
 - realtime resampling
 - advanced arranger/song mode
 - plugin architecture
 - Electron/native desktop version
 - WASAPI loopback recording
 - external MIDI sync
 - true MPC live performance workflow
-

Final Notes

The project reached the point where it already feels like a real instrument instead of a UI mockup.

Biggest architectural wins:

- bank-aware sequencing
- real program ownership
- PPQ timing stabilization
- song convert offset fix
- removal of fake workflow assumptions

Main risk going forward: accidental refactors breaking established MPC workflow behavior.

Treat sequencing/program architecture as foundational.